

1. Python & VS Code 설치

1) Python 설치

본 과정에서는 Python 3.12.10 version 설치

- 단순히 최신 버전을 사용하는 것보다 개발 환경의 안정성과 라이브러리 호환성과 많은 레퍼런스들을 고려하여 3.12.10 버전 선택

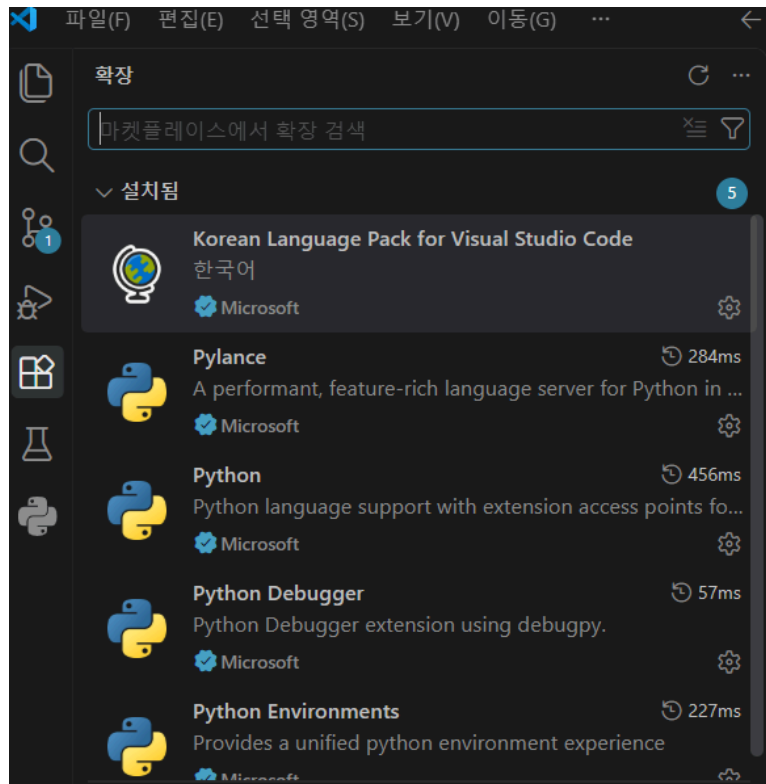
✓ 설치 확인

1. 시스템 환경 변수 편집 → 환경 변수 → Path도 제대로 등록되어있는지 확인하여 `python` 명령어를 바로 사용할 수 있도록 한다.
2. CMD 창을 실행
3. `Python --version` 명령어 입력
4. `Python 3.12.10` 과 같은 Python 버전이 출력되면 정상적으로 설치 완료

2) VS Code 설치

VS Code 설치 후, 한국어 환경과 Python 개발을 위해 확장 프로그램 설치 (익스텐션 설치)

1. `Korean Language Pack` - VS Code 한국어 환경 설정
2. `Python` - VS Code에서 Python 개발 지원



3. PPython이 정상적으로 설치되었는지 확인하기 위해 CMD에서 설치한 버전과 동일한 버전이 출력되는지 확인 → 똑같이 **Python 3.12.10** 출력되면 정상적으로 설치 완료

4. 프로젝트 폴더 만들기 → 프로젝트 생성 → 가상환경 생성

📁 프로젝트 폴더

└─ 📁 py

└─ 📁 가상환경

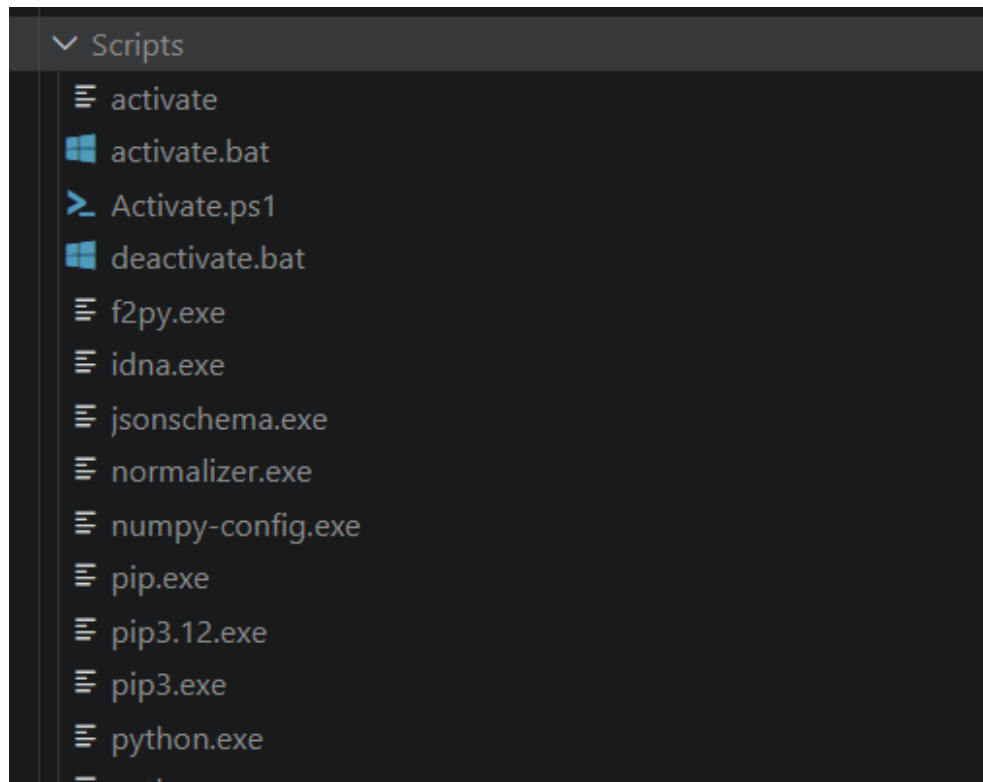
• 가상 환경을 만드는 이유

프로젝트별로 독립된 Python 개발 환경을 만들어 라이브러리 간 충돌 방지

1 python -m venv .venv을 넣어 venv 가상 환경 생성

2 가상 환경 활성화를 위해 activate.bat의 경로를 복사 후 cmd 창에 입력

가상 환경 비활성화를 하고 싶은 경우 : deactivate.bat 경로 복사 후 cmd 창에 입력



3 다음과 같이 현재 작업 중인 폴더 앞에 가상환경 활성화 됨을 알 수 있음

(.thirdenv) 는 본 과정에서 생성한 가상환경의 이름을 (thirdenv) 로 지정했기 때
문

```
(.thirdenv) D:\yujin0902\ex_git_0904>
```

2. Python 문법 1일차

1) 변수, 출력, 주석

변수에 값을 저장하고 `print()` 를 이용하여 결과를 출력할 수 있다.

- 변수: 데이터를 저장하는 공간
- `print()`: 변수나 값을 출력하는 함수
- 문자열은 `" "` 또는 `' '` 로 표현
- 숫자는 따옴표 없이 표현

💡 이때, `print()` 에서 `sep` 을 사용하면 여러 값 사이에 원하는 문자를 넣을 수 있다.

```
print("파이썬 기본 문법", 8, sep=", 시간:")
```

출력: 파이썬 기본 문법, 시간:8

- 주석은 #을 이용해서 사용
- 전체 주석을 하고 싶은 경우 ctrl+/를 통해 가능

2) 데이터 형식 (Type)

변수에 저장된 데이터의 자료형을 확인할 수 있다.

- `str` → 문자열
- `int` → 정수
- `float` → 실수
- `bool` → 참/거짓
- `list` → 리스트
- `dict` → 딕셔너리

📌 `type()` 을 사용하면 변수에 저장된 데이터의 **자료형(Type)**을 확인할 수 있다.

```
print(type(x))
```

3) for 반복문 과 문자열 List

- for문 : 여러 데이터를 하나씩 꺼내 반복적으로 처리할 때 사용한다.

```
for i in 5:  
    print(i)
```

출력: 1 2 3 4 5

- List: 여러 개의 데이터를 순서대로 저장하고 관리하는 자료구조

```
list= [1,2,3,4,5]  
print(list[0])
```

출력:1

📌 인덱스는 0부터 시작

- 문자열: `" "` 또는 `' '` 안에 있는 문자열은 배열
- 문자열의 길이를 구하기 위해서는 `len()` 사용

```
a = "Hello, World!"  
print(a[1])  
print(len(a))
```

출력: e, 13

4) 함수 Function

함수: 반복해서 사용하거나 하나의 기능으로 묶고 싶은 코드를 함수로 정의하여 재사용

- `def` 를 사용하여 함수를 정의하고, 함수 이름을 호출하여 실행

```
x="awesome"  
  
def myfunc():  
    x="111"  
    print("Python is " + x)  
  
myfunc()  
print("music is "+ x)
```

출력: Python is 111
music is awesome

5) NumPy

수치 계산과 배열 연산을 쉽게 할 수 있도록 도와주는 라이브러리

```
import numpy as np  
  
arr=np.array([1,2,3,4,5])  
print(arr)
```

📌 `np.array()` 를 이용해 NumPy 배열을 생성할 수 있다.